

FORMAL PROJECT REPORT

JobPortal: A Full-Stack Career Opportunity Platform

Date: April 23, 2026

Prepared By: Lakshay Shakya(24BCS10805) , 24BCS_KRG_616B

Subject: Technical Overview and Implementation Report of the JobPortal Project

1. Executive Summary

The JobPortal project is a comprehensive full-stack web application designed to bridge the gap between job seekers and employers. Built using modern web technologies—**React.js** for the frontend, **Spring Boot** for the backend, and **MySQL** for persistent storage—it provides a seamless, high-performance environment for managing job listings. This report outlines the system architecture, core functionalities, and the technical implementation of the platform.

2. Introduction

In the modern digital landscape, the efficiency of recruitment processes depends heavily on the accessibility and clarity of job data. The JobPortal project was initiated to create a robust, responsive, and scalable platform where:

- Users can browse latest job openings with real-time filtering.
- Recruiters can post new opportunities through an intuitive interface.
- Data integrity is maintained through a secure relational database.

The project transitions from a static data model to a dynamic, API-driven architecture, ensuring that the platform can scale with user growth.

3. System Architecture

The application follows a standard **Client-Server Architecture** with a clear separation of concerns between the presentation layer and the data processing layer.

3.1 Frontend Architecture

- **Framework:** React.js (v18+) managed by Vite for optimized build cycles.
- **Styling:** Vanilla CSS combined with Tailwind CSS for rapid UI development and a "glassmorphism" aesthetic.
- **State Management:** React Hooks (useState, useEffect) manage the local component states and data synchronization with the server.
- **Dynamic Routing:** Handled internally to maintain a single-page application (SPA) experience.

3.2 Backend Architecture

- **Framework:** Spring Boot (Java) providing a RESTful API layer.
- **Data Access:** Spring Data JPA with Hibernate for Object-Relational Mapping (ORM).
- **Communication:** Maven handles dependency management and build processes.
- **Proxy Configuration:** Vite is configured to proxy API requests to the Spring Boot server, resolving CORS issues during development.

3.3 Database Layer

- **System:** MySQL.
- **Persistence:** The application uses two primary tables: jobs (storing core job details) and job_skills (a collection table linked to jobs using a one-to-many relationship).

4. Technical Implementation & Data Flow

The platform operates through a well-defined data flow that ensures consistency across the stack.

4.1 Data Retrieval & Filtering

When the application loads, the frontend triggers an asynchronous fetchJobs function:

1. **Request:** A GET request is sent to /api/jobs.
2. **Backend Processing:** The JobController receives the request and parses optional query parameters.
3. **Repository Layer:** The JobRepository executes a custom JPQL query:

```
@Query("SELECT j FROM Job j WHERE (:role IS NULL OR j.title LIKE %:role%) AND ...")
```

Response: Data is returned as a JSON array and rendered by the `JobCard` components.

4.2 Job Creation Process

Posting a job involves a multi-step validation and persistence cycle:

- **User Interaction:** The user opens the `PostJobModal` and fills in details.
- **Payload Construction:** The React frontend constructs a JSON object matching the Job entity schema.
- **API Call:** A `POST` request is sent to `/api/jobs`.
- **Persistence:** The Spring Boot backend validates and saves the entry using `jobRepository.save(job)`.

5. Core Features

5.1 Dynamic Search & Filtering

The platform features a "Search Bar" that allows users to narrow down listings by Role, Job Type (Full-time, Part-time, Contract), Location, and Experience level.

5.2 Responsive & High-Aesthetic UI

- **Glassmorphism:** Elegant semi-transparent cards.
- **Micro-animations:** Hover effects and smooth transitions.
- **Loading States:** Shimmer/Spinning indicators for improved UX.

5.3 Robust Backend Integration

The system handles database exceptions gracefully, displaying clear error states with "Retry" mechanisms if the backend is unreachable.

6. Challenges & Solutions

- **Database Connection:** Resolved authentication issues by configuring specific privileges in `application.properties`.
- **CORS Management:** Resolved using a Vite proxy and `@CrossOrigin` annotations in Spring Boot.
- **Data Types:** Handled complex skill lists using `ElementCollections` in JPA to maintain normalization.

7. Conclusion & Future Roadmap

The JobPortal project successfully demonstrates the integration of a React frontend with a Java-based backend. Future enhancements include User Authentication (JWT), Applicant Tracking, and an Analytics Dashboard using Chart.js.

End of Report